

Git Commands

Chapter 1 — Basics

1.1 Init repo — Repository စတင်ဖန်တီးခြင်း

Project တစ်ခုကို Git နဲ့ စတင်အသုံးပြုမယ်ဆိုရင် အရင်ဆုံး **Git Repository** တစ်ခုကို ဖန်တီးရပါတယ်။

Create an empty repo - Repository အလွတ်တစ်ခု ဖန်တီးခြင်း

Git Repository အသစ်တစ်ခု ဖန်တီးဖို့ git init command ကို အသုံးပြုပါတယ်။

```
git init
```

Command ကို အောင်မြင်စွာ run လုပ်ပြီးရင်—

Initialized empty Git repository in /tmp/repo/.git/ လို့ ပြပါလိမ့်မယ်။

ဒါဟာ /tmp/repo/ ထဲမှာ **Git Repository အသစ်တစ်ခု ဖန်တီးပြီးပြီ** ဆိုတဲ့ အဓိပ္ပာယ်ပါ။

Set user name and email - Repository အတွက် User Name နဲ့ Email သတ်မှတ်ခြင်း

Git Commit တွေမှာ အသုံးပြုမယ့် User Name နဲ့ Email ကို သတ်မှတ်နိုင်ပါတယ်။

```
git config user.email alice@example.com
```

```
git config user.name "Alice Zakas"
```

Show user and repo configs - User နဲ့ Repository Configuration တွေကို ကြည့်ခြင်း

သတ်မှတ်ထားတဲ့ Git configuration တွေကို ကြည့်ဖို့—

```
git config --list --show-origin
```

ကို အသုံးပြုနိုင်ပါတယ်။

ဒီ command က **User configuration** နဲ့ **Repository configuration** တွေ ဘယ်နေရာကနေ သတ်မှတ်ထားတယ်ဆိုတာပါ ပြပေးပါတယ်။

1.2 Add file — File ထည့်ခြင်း

File အသစ်တစ်ခု ဖန်တီးပြီး Staging Area ထဲ ထည့်ခြင်း

File အသစ်တစ်ခုကို ဖန်တီးပြီး Git ရဲ့ **staging area** ထဲ ထည့်ဖို့ git add command ကို အသုံးပြုပါတယ်။

```
echo "git is awesome" > message.txt
```

```
git add message.txt
```

ဒီအဆင့်မှာ message.txt ကို Git က **commit** လုပ်ဖို့အတွက် **staging area** ထဲမှာ ပြင်ဆင်ထားပါတယ်။

View changes in the staging area- Staging Area ထဲက ပြောင်းလဲမှုကို ကြည့်ခြင်း

Staging area ထဲမှာ ဘာတွေပြောင်းလဲထားတယ်ဆိုတာ ကြည့်ဖို့—

```
git diff --cached
```

ကို အသုံးပြုနိုင်ပါတယ်။

Commit to the local repo - Local Repository ထဲကို Commit လုပ်ခြင်း

Staging area ထဲမှာ ပြင်ဆင်ထားတဲ့ File ကို Local Repository ထဲမှာ အမြဲတမ်းမှတ်တမ်းတင်ဖို့ git commit ကို အသုံးပြုပါတယ်။

```
git commit -m "add message"
```

အောင်မြင်ရင်—

```
[main (root-commit) 3a2bd8f] add message
```

```
1 file changed, 1 insertion(+)
```

```
create mode 100644 message.txt
```

လို့ ပြပါလိမ့်မယ်။

အဓိကအားဖြင့် ဒီအပိုင်းမှာ—

Create file → git add → git diff --cached → git commit

ဆိုတဲ့ လုပ်ဆောင်မှုအစဉ်ကို လေ့လာရပါတယ်။

1.3 Edit file — File ပြင်ဆင်ခြင်း

Edit the previously committed file - အရင်က Commit လုပ်ထားတဲ့ File ကို ပြင်ဆင်ခြင်း

အရင်က Commit လုပ်ထားပြီးသား message.txt File ကို ပြန်လည်ပြင်ဆင်နိုင်ပါတယ်။

```
echo "git is awesome" > message.txt
```

ဒီ Command နဲ့ message.txt ထဲက စာသားကို ပြင်ဆင်လိုက်ပါတယ်။

View local changes - Local Changes တွေကို ကြည့်ခြင်း

File ကို ပြင်ပြီးတဲ့နောက် ဘာတွေပြောင်းလဲသွားတယ်ဆိုတာ ကြည့်ဖို့—

```
git diff
```

ကို အသုံးပြုနိုင်ပါတယ်။

ဥပမာ—

```
- git is awesom  
+ git is awesome
```

ဒီမှာ - က အရင်စာသားဖြစ်ပြီး + က ပြင်ဆင်ပြီးတဲ့ စာသား ဖြစ်ပါတယ်။

Add modified files and commit in one command

(ပြင်ဆင်ထားတဲ့ File ကို Add လုပ်ပြီး Commit တစ်ခါတည်း ပြုလုပ်ခြင်း)

အရင်က Commit လုပ်ထားပြီးသား File တွေရဲ့ ပြင်ဆင်မှုတွေကို git commit -am နဲ့ တစ်ခါတည်း Add လုပ်ပြီး Commit လုပ်နိုင်ပါတယ်။

```
git commit -am "edit message"
```

အောင်မြင်ရင်—

```
[main ecdeb79] edit message  
1 file changed, 1 insertion(+), 1 deletion(-)
```

လို့ ပြပါလိမ့်မယ်။

မှတ်ရန်: -a က အရင်က Commit လုပ်ထားပြီးသား File တွေရဲ့ ပြင်ဆင်မှုကိုပဲ Add လုပ်ပေးပါတယ်။

File အသစ်တွေကိုတော့ -a တစ်ခုတည်းနဲ့ Add မလုပ်ပေးပါဘူး။

1.4 Rename file – File အမည်ပြောင်းခြင်း

Rename the previously committed file- အရင်က Commit လုပ်ထားတဲ့ File ကို အမည်ပြောင်းခြင်း

အရင်က Commit လုပ်ထားတဲ့ message.txt File ကို praise.txt လို့ အမည်ပြောင်းဖို့ git mv command ကို အသုံးပြုပါတယ်။

```
git mv message.txt praise.txt
```

ဒီအဆင့်မှာ File အမည်ပြောင်းတဲ့ အပြောင်းအလဲဟာ **staging area** ထဲကို တစ်ခါတည်း ရောက်ရှိသွားပါတယ်။

ဒါကြောင့် ပုံမှန် git diff နဲ့ ကြည့်ရင် ပြောင်းလဲမှုကို မတွေ့ရပါဘူး။ Staging area ထဲက ပြောင်းလဲမှုကို ကြည့်ဖို့—

```
git diff --cached
```

ကို အသုံးပြုရပါတယ်။

ဥပမာ—

```
diff --git a/message.txt b/praise.txt
```

```
similarity index 100%
```

```
rename from message.txt
```

```
rename to praise.txt
```

ဒီမှာ Git က message.txt ကို praise.txt အဖြစ် **File အမည်ပြောင်းထားတာ** ကို ပြပေးပါတယ်။

Commit the change- File အမည်ပြောင်းထားမှုကို Commit လုပ်ခြင်း

အမည်ပြောင်းထားတဲ့ အပြောင်းအလဲကို Repository ထဲမှာ မှတ်တမ်းတင်ဖို့—

```
git commit -m "rename message.txt"
```

အောင်မြင်ရင်—

```
[main d768287] rename message.txt
```

```
1 file changed, 0 insertions(+), 0 deletions(-)
```

```
rename message.txt => praise.txt (100%)
```

လို့ ပြပါလိမ့်မယ်။

အဓိကလုပ်ဆောင်မှု

```
message.txt
```

↓

```
git mv message.txt praise.txt
```

↓

```
git diff --cached
```

↓

```
git commit -m "rename message.txt"
```

1.5 Delete file — File ဖျက်ခြင်း

Delete the previously committed file- အရင်က Commit လုပ်ထားတဲ့ File ကို ဖျက်ခြင်း

အရင်က Commit လုပ်ထားတဲ့ File တစ်ခုကို ဖျက်ဖို့ git rm command ကို အသုံးပြုပါတယ်။

```
git rm message.txt
```

ဒီ Command ကို အသုံးပြုလိုက်တဲ့အခါ message.txt ကို ဖျက်လိုက်ပြီး အဲဒီအပြောင်းအလဲဟာ staging area ထဲကို တစ်ခါတည်း ရောက်ရှိသွားပါတယ်။

View the staged change- Staging Area ထဲက အပြောင်းအလဲကို ကြည့်ခြင်း

File ဖျက်ထားတဲ့ အပြောင်းအလဲကို ကြည့်ဖို့—

```
git diff --cached
```

ကို အသုံးပြုနိုင်ပါတယ်။

ဥပမာ—

```
diff --git a/message.txt b/message.txt
deleted file mode 100644
--- a/message.txt
+++ /dev/null
@@ -1 +0,0 @@
-git is awesome
```

ဒီမှာ message.txt ကို ဖျက်ထားကြောင်း Git က ပြပေးပါတယ်။

Commit the change - File ဖျက်ထားမှုကို Commit လုပ်ခြင်း

File ဖျက်ထားတဲ့ အပြောင်းအလဲကို Repository ထဲမှာ မှတ်တမ်းတင်ဖို့—

```
git commit -m "delete message.txt"
```

အောင်မြင်ရင်—

```
[main 6a2d19b] delete message.txt
1 file changed, 1 deletion(-)
delete mode 100644 message.txt
```

လို့ ပြပါလိမ့်မယ်။

အဓိကလုပ်ဆောင်မှု

message.txt

↓

```
git rm message.txt
```

↓

```
git diff --cached
```

↓

```
git commit -m "delete message.txt"
```

1.6 Status – လက်ရှိအခြေအနေကို ကြည့်ခြင်း

Edit the previously committed file and add the changes to the staging area

(အရင်က Commit လုပ်ထားတဲ့ File ကို ပြင်ပြီး အပြောင်းအလဲကို Staging Area ထဲ ထည့်ခြင်း)

အရင်က Commit လုပ်ထားတဲ့ message.txt ကို ပြင်ဆင်ပြီး Staging Area ထဲ ထည့်ပါမယ်။

```
echo "git is awesome" > message.txt
```

```
git add message.txt
```

Create a new file - File အသစ်တစ်ခု ဖန်တီးခြင်း

File အသစ်တစ်ခုကိုလည်း ဖန်တီးနိုင်ပါတယ်။

```
echo "git is great" > praise.txt
```

ဒီအချိန်မှာ message.txt က staging area ထဲ ရောက်နေပြီး praise.txt ကတော့ tracked မဖြစ်သေးတဲ့ File ဖြစ်ပါတယ်။

Show the working tree status - Working Tree ရဲ့ လက်ရှိအခြေအနေကို ကြည့်ခြင်း

Repository ရဲ့ လက်ရှိအခြေအနေကို ကြည့်ဖို့—

```
git status
```

ကို အသုံးပြုပါတယ်။

ဥပမာ—

On branch main

Changes to be committed:

modified: message.txt

Untracked files:

praise.txt

ဒီမှာ—

- **Changes to be committed** → message.txt ကို ပြင်ပြီး Staging Area ထဲ ထည့်ထားပါတယ်။
- **Untracked files** → praise.txt က File အသစ်ဖြစ်ပြီး Git က မခြေရာခံသေးပါဘူး။

အဓိက မှတ်ရန်

git status က လက်ရှိ Repository မှာ ဘာတွေပြောင်းလဲထားလဲ၊ ဘာတွေကို Commit လုပ်ဖို့ ပြင်ဆင်ထားလဲ၊ ဘာတွေက

Untracked ဖြစ်နေသလဲ ဆိုတာကို စစ်ဆေးဖို့ အသုံးပြုတဲ့ Command ဖြစ်ပါတယ်။

1.7 Log – Commit မှတ်တမ်းကို ကြည့်ခြင်း

Show commits - Commit တွေကို ကြည့်ခြင်း

Repository ထဲမှာ ပြုလုပ်ထားတဲ့ Commit တွေကို ကြည့်ဖို့ git log command ကို အသုံးပြုပါတယ်။

`git log`

ဒီ Command ကို အသုံးပြုရင် Commit တစ်ခုချင်းစီရဲ့—

- Commit hash
- Author
- Date
- Commit message

တို့ကို ပြပေးပါတယ်။

ဥပမာ—

```
commit ecdeb79aad4565d8d7d725678ffadc48b3cdec52
Author: sandbox <sandbox@example.com>
Date: Thu Mar 14 15:00:00 2024 +0000
```

edit message

```
commit 3a2bd8f0929c605193120bd1ad12732f49457e99
Author: sandbox <sandbox@example.com>
Date: Thu Mar 14 15:00:00 2024 +0000
```

add message

Show only the commit message and short hash (Commit Message နဲ့ Short Hash ကိုပဲ ကြည့်ခြင်း)

Commit မှတ်တမ်းကို ပိုပြီးတိုတောင်းတဲ့ပုံစံနဲ့ ကြည့်ချင်ရင်—

`git log --oneline`

ကို အသုံးပြုနိုင်ပါတယ်။

ဥပမာ—

```
ecdeb79 edit message
3a2bd8f add message
```

ဒီလိုဆိုရင် **Commit ရဲ့ Short Hash နဲ့ Commit Message ကိုပဲ** တစ်ကြောင်းစီ ပြပေးမှာ ဖြစ်ပါတယ်။

အဓိက မှတ်ရန်

`git log`

→ Commit အသေးစိတ်မှတ်တမ်းကို ကြည့်ရန်

`git log --oneline`

→ Commit မှတ်တမ်းကို တိုတောင်းတဲ့ပုံစံနဲ့ ကြည့်ရန်

1.8 Show commit – သတ်မှတ်ထားသော Commit ကို ကြည့်ခြင်း

Show a specific commit - သတ်မှတ်ထားတဲ့ Commit တစ်ခုကို ကြည့်ခြင်း

Commit တစ်ခုရဲ့ အသေးစိတ်အချက်အလက်နဲ့ အဲဒီ Commit မှာ ပြုလုပ်ထားတဲ့ ပြောင်းလဲမှုတွေကို ကြည့်ဖို့ git show command ကို အသုံးပြုပါတယ်။

အရင်ဆုံး Commit ရဲ့ hash ကို git log ကနေ ရယူနိုင်ပါတယ်။

```
git log --oneline
```

ဥပမာ—

```
ecdeb79 edit message
```

```
3a2bd8f add message
```

ပြီးရင် သတ်မှတ်ထားတဲ့ Commit ကို—

```
git show ecdeb79
```

ဆိုပြီး ကြည့်နိုင်ပါတယ်။

ဒီ Command က အဲဒီ Commit ရဲ့—

- Commit Hash
- Author
- Date
- Commit Message
- ဘယ် File တွေ ပြောင်းလဲခဲ့သလဲ
- ဘာတွေ ထည့်ခဲ့သလဲ / ဖျက်ခဲ့သလဲ

စတဲ့ အချက်အလက်တွေကို ပြပေးပါတယ်။

Show only the commit summary - Commit ရဲ့ အကျဉ်းချုပ်ကိုသာ ကြည့်ခြင်း

Commit တစ်ခုမှာ ဘယ် File တွေ ပြောင်းလဲခဲ့တယ်ဆိုတာကို အကျဉ်းချုပ်ပုံစံနဲ့ ကြည့်ချင်ရင်—

```
git show --stat ecdeb79
```

ကို အသုံးပြုနိုင်ပါတယ်။

အဓိက မှတ်ရန်

```
git show <commit>
```

→ သတ်မှတ်ထားတဲ့ Commit ရဲ့ အသေးစိတ်အကြောင်းအရာနဲ့ ပြောင်းလဲမှုတွေကို ကြည့်ရန်။

```
git show --stat <commit>
```

→ Commit မှာ ပြောင်းလဲခဲ့တဲ့ File တွေကို အကျဉ်းချုပ်ကြည့်ရန်။

1.9 Search repo – Repository အတွင်း ရှာဖွေခြင်း

Search for text in the repository - Repository အတွင်းရှိ စာသားကို ရှာဖွေခြင်း

Repository ထဲမှာ သတ်မှတ်ထားတဲ့ စာသားတစ်ခုကို ရှာဖွေဖို့ git grep command ကို အသုံးပြုနိုင်ပါတယ်။

ဥပမာ—

```
git grep "awesome"
```

ဒီ Command က Repository ထဲမှာ awesome ဆိုတဲ့ စာသား ပါဝင်တဲ့နေရာတွေကို ရှာဖွေပေးပါတယ်။

Search in a specific commit - သတ်မှတ်ထားတဲ့ Commit အတွင်းမှာ ရှာဖွေခြင်း

Commit တစ်ခုအတွင်းမှာ စာသားကို ရှာချင်ရင် Commit hash ကို သတ်မှတ်ပြီး ရှာနိုင်ပါတယ်။

```
git grep "awesome" <commit>
```

ဒီလိုလုပ်ခြင်းအားဖြင့် လက်ရှိ Working Tree ထဲမှာသာမက သတ်မှတ်ထားတဲ့ Commit ထဲက အကြောင်းအရာတွေကိုပါ ရှာဖွေနိုင်ပါတယ်။

Search for a file - Repository ထဲမှာ File ကို ရှာဖွေခြင်း

Repository ရဲ့ မှတ်တမ်းထဲမှာ File တစ်ခု ဘယ်လိုရှိခဲ့တယ်ဆိုတာကို ရှာဖွေဖို့ Git ရဲ့ search commands တွေကို အသုံးပြုနိုင်ပါတယ်။

အဓိက မှတ်ရန်

```
git grep "စာသား"
```

→ Repository ထဲမှာ သတ်မှတ်ထားတဲ့ စာသားကို ရှာရန်။

```
git grep "စာသား" <commit>
```

→ သတ်မှတ်ထားတဲ့ Commit အတွင်းမှာ စာသားကို ရှာရန်။

ဒီအပိုင်းရဲ့ အဓိကရည်ရွယ်ချက်က **Repository ထဲက လိုအပ်တဲ့ အကြောင်းအရာကို အလွယ်တကူ ရှာဖွေနိုင်အောင် Git ရဲ့ search capability ကို အသုံးပြုတတ်စေဖို့** ဖြစ်ပါတယ်။

Chapter 2 — Branch and merge

2.1 Branch — Branch ခွဲခြင်း

Show branches - ရှိပြီးသား Branch တွေကို ကြည့်ခြင်း

Repository ထဲမှာ လက်ရှိရှိနေတဲ့ Branch တွေကို ကြည့်ဖို့—

`git branch` အသုံးပြုနိုင်ပါတယ်။

အစမှာ main Branch တစ်ခုပဲ ရှိနေပါတယ်။

* main

(*) သင်္ကေတက လက်ရှိအသုံးပြုနေတဲ့ Branch ကို ဆိုလိုပါတယ်။

Create a new branch - Branch အသစ်တစ်ခု ဖန်တီးခြင်း

Branch အသစ်တစ်ခု ဖန်တီးဖို့—

`git branch ohmypy` ကို အသုံးပြုပါတယ်။

ဒီ Command က ohmypy ဆိုတဲ့ Branch အသစ်ကို ဖန်တီးပေးပါတယ်။

Switch to the new branch - Branch အသစ်ကို ပြောင်းသုံးခြင်း

ဖန်တီးထားတဲ့ ohmypy Branch ကို ပြောင်းသုံးဖို့—

`git switch ohmypy`

အသုံးပြုပါတယ်။ အောင်မြင်ရင်—

Switched to branch 'ohmypy'

လို့ ပြပါလိမ့်မယ်။ ပြီးရင်—

`git branch`

နဲ့ Branch တွေကို ပြန်ကြည့်ရင်—

main

* ohmypy လို့ ပြပါလိမ့်မယ်။

ဒီမှာ * ohmypy ဖြစ်နေတဲ့အတွက် လက်ရှိအလုပ်လုပ်နေတဲ့ Branch က ohmypy ဖြစ်သွားပြီ ဆိုတာ သိနိုင်ပါတယ်။

Add and commit a file - File ထည့်ပြီး Commit လုပ်ခြင်း

ohmypy Branch ထဲမှာ File အသစ်တစ်ခု ဖန်တီးပြီး Commit လုပ်နိုင်ပါတယ်။

`echo "print('git is awesome')" > ohmy.py`

`git add ohmy.py`

`git commit -m "ohmy.py"`

ဒီလိုနဲ့ ohmy.py File ရဲ့ Commit ဟာ ohmypy Branch ထဲမှာ ရှိလာပါမယ်။

အဓိက မှတ်ရန်

`git branch` → Branch တွေကို ကြည့်ရန်

`git branch ohmypy` → Branch အသစ် ဖန်တီးရန်

`git switch ohmypy` → Branch ပြောင်းရန်

2.2 Merge – Branch များကို ပေါင်းစည်းခြင်း

Show commits from all branches - Branch အားလုံးရဲ့ Commit မှတ်တမ်းကို ကြည့်ခြင်း

Branch နှစ်ခုမှာ သီးခြား Commit တွေ ရှိနေတဲ့အခါ Branch အားလုံးရဲ့ Commit history ကို ကြည့်ဖို့—

```
git log --all --oneline --graph
```

ကို အသုံးပြုနိုင်ပါတယ်။

ဥပမာ—

```
* ecdeb79 edit message
```

```
| * a715138 ohmy.py
```

```
|/
```

```
* 3a2bd8f add message
```

ဒီမှာ main Branch မှာ Commit နှစ်ခုရှိပြီး ohmypy Branch မှာ Commit တစ်ခု ထပ်ရှိနေတာကို တွေ့နိုင်ပါတယ်။

Merge the branch into main - Branch ကို main ထဲ ပေါင်းစည်းခြင်း

အခု main Branch ကို ပြန်သွားပြီး ohmypy Branch ကို main ထဲ ပေါင်းစည်းပါမယ်။

```
git merge ohmypy
```

Merge လုပ်တဲ့အခါ Conflict မရှိရင် Git က Merge Commit ကို အလိုအလျောက် ဖန်တီးပေးပါတယ်။

Merge made by the 'ort' strategy.

```
ohmy.py | 1 +
```

```
1 file changed, 1 insertion(+)
```

```
create mode 100644 ohmy.py
```

View the new commit history- Merge ပြီးနောက် Commit History ကို ကြည့်ခြင်း

Merge ပြီးတဲ့နောက် Commit history ကို ပြန်ကြည့်နိုင်ပါတယ်။

```
git log --all --oneline --graph
```

ဥပမာ—

```
* 7d5ac4f Merge branch 'ohmypy'
```

```
| \
```

```
| * a715138 ohmy.py
```

```
* | ecdeb79 edit message
```

```
|/
```

```
* 3a2bd8f add message
```

ဒီလိုနဲ့ ohmypy Branch ထဲက Commit ကို main Branch နဲ့ ပေါင်းစည်းပြီးသား ဖြစ်သွားပါတယ်။

အဓိက မှတ်ရန်

```
git merge ohmypy
```

→ ohmypy Branch ထဲက အပြောင်းအလဲတွေကို လက်ရှိ Branch ထဲ ပေါင်းစည်းရန်။

```
git log --all --oneline --graph
```

→ Branch အားလုံးရဲ့ Commit History နဲ့ Merge ဖြစ်ပုံကို ကြည့်ရန်။

2.3 Rebase — Branch ကို ပြန်လည်အခြေခံတင်ခြင်း

Show commits from all branches - Branch အားလုံးရဲ့ Commit မှတ်တမ်းကို ကြည့်ခြင်း

main Branch မှာ Commit နှစ်ခုရှိပြီး ohmypy Branch မှာ Commit တစ်ခုရှိနေတဲ့အခြေအနေကို ကြည့်နိုင်ပါတယ်။

```
git log --all --oneline --graph
```

ဥပမာ—

```
* ecdeb79 edit message
```

```
| * a715138 ohmy.py
```

```
|/
```

```
* 3a2bd8f add message
```

ဒီမှာ main နဲ့ ohmypy Branch တွေဟာ သီးခြားခွဲထွက်နေပါတယ်။

Rebase the branch - Branch ကို Rebase လုပ်ခြင်း

ဥပမာ main Branch ပေါ်မှာ ရှိနေပြီး ohmypy Branch ကို အခြေခံပြီး main ရဲ့ Commit တွေကို ပြန်လည်တင်ပေးပါတယ်။

```
git rebase ohmypy
```

အောင်မြင်ရင်—

```
Rebasing (1/1)
```

```
Successfully rebased and updated refs/heads/main.
```

လို့ ပြပါလိမ့်မယ်။

View the new commit history - Rebase ပြီးနောက် Commit History ကို ကြည့်ခြင်း

Rebase ပြီးသွားတဲ့အခါ Commit History ဟာ `git merge` လုပ်တဲ့အခါလို Merge Commit ပါတဲ့ ပုံစံမဟုတ်တော့ဘဲ တစ်

ကြောင်းတည်း (linear history) ဖြစ်လာပါတယ်။

Rebase ပြီးတဲ့ Commit History ဟာ Merge လုပ်ထားတဲ့ History ထက် ပိုပြီး linear ဖြစ်ပါတယ်။

မှတ်ချက် - Rebase က Git History ကို ပြန်ရေးသား (rewrite) ပါတယ်။

ဒါကြောင့် Remote Repository ကို Push လုပ်ပြီးသား Branch တွေကို Rebase မလုပ်တာ ပိုကောင်းပါတယ်။

အဓိက မှတ်ရန်

```
git rebase ohmypy
```

→ ohmypy Branch ကို အခြေခံပြီး လက်ရှိ Branch ရဲ့ Commit History ကို ပြန်လည်စီစဉ်ရန်။

Merge က Merge Commit တစ်ခု ဖန်တီးနိုင်ပေမယ့် **Rebase** က Commit History ကို ပြန်ရေးပြီး **Linear History** ဖြစ်

အောင် လုပ်ပေးပါတယ်။

2.4 Squash – Commit များကို ပေါင်းစည်းခြင်း

Branch တစ်ခုထဲက Commit အားလုံးကို Commit တစ်ခုတည်းအဖြစ် ပေါင်းစည်းခြင်း

တစ်ခါတလေ Branch တစ်ခုမှာ ပြုလုပ်ထားတဲ့ Commit အားလုံးကို တစ်ခုချင်းစီမထားဘဲ Commit တစ်ခုတည်းအဖြစ်

ပေါင်းစည်းပြီး main Branch ထဲကို Merge လုပ်ချင်နိုင်ပါတယ်။ ဒီလိုလုပ်တာကို Squash လို့ ခေါ်ပါတယ်။

ဥပမာမှာ ohmypy Branch ထဲမှာ Commit ၃ ခု ရှိပါတယ်—

b9a7d0f ohmy.lua

5ca4d55 ohmy.sh

ဒီ Commit ၃ ခုကို Commit တစ်ခုတည်းအဖြစ် ပေါင်းစည်းပါမယ်။

Switch to the branch - သက်ဆိုင်ရာ Branch ကို ပြောင်းခြင်း

`git switch ohmypy`

ပြီးရင် ohmypy Branch ထဲက အပြောင်းအလဲတွေကို Squash လုပ်ဖို့—

`git merge --squash main` ကို အသုံးပြုပါတယ်။

ဒီအချိန်မှာ Git က အပြောင်းအလဲတွေကို တစ်စုတစ်စည်းတည်းအဖြစ် Working Directory ထဲမှာ ပြင်ဆင်ပေးပေမယ့်

Commit ကို အလိုအလျောက် မလုပ်ပေးပါဘူး။

Commit the combined changes - ပေါင်းစည်းထားတဲ့ အပြောင်းအလဲတွေကို Commit လုပ်ခြင်း

Squash ပြီးတဲ့နောက် ပေါင်းစည်းထားတဲ့ အပြောင်းအလဲတွေကို Commit လုပ်ဖို့—

`git commit -m "ohmy[py,sh,lua]"`

Commit တစ်ခု ဖန်တီးထားပါတယ်—

[ohmypy 4f2a17f] ohmy[py,sh,lua]

ဒါကြောင့် အရင်က Commit ၃ ခုနဲ့ ပြုလုပ်ထားတဲ့ အပြောင်းအလဲတွေကို Commit တစ်ခုတည်းအဖြစ် စုစည်းထားနိုင်ပါတယ်။

Merge the squashed branch into main- Squash လုပ်ထားတဲ့ Branch ကို main ထဲ ပေါင်းစည်းခြင်း

main Branch ကို ပြန်ပြောင်းပြီး ohmypy Branch ကို Merge လုပ်ပါတယ်။

`git switch main`

`git merge --no-ff ohmypy -m "ohmy[py,sh,lua]"`

ဒီလိုလုပ်ပြီးရင် ohmypy Branch ထဲက Commit ၃ ခုဟာ main ဘက်မှာ Commit တစ်ခုတည်းအဖြစ် ပေါင်းစည်းထားတာကို Commit History မှာ တွေ့နိုင်ပါတယ်။

အဓိက မှတ်ရန်

Squash ရဲ့ အဓိကရည်ရွယ်ချက်က—

Commit 1

Commit 2 —————→ Commit တစ်ခု

Commit 3

ဆိုပြီး Branch တစ်ခုထဲက Commit အများကြီးကို Commit တစ်ခုတည်းအဖြစ် စုစည်းပေးခြင်း ဖြစ်ပါတယ်။

2.5 Cherry-pick — သတ်မှတ်ထားသော Commit ကို ရွေးယူခြင်း

အခြား Branch ထဲက သတ်မှတ်ထားတဲ့ Commit တစ်ခုကို ရွေးယူခြင်း

တစ်ခါတလေ အခြား Branch ထဲမှာ ပြင်ဆင်ထားတဲ့ အပြောင်းအလဲတစ်ခုကို Branch တစ်ခုလုံးကို Merge မလုပ်ဘဲ လက်ရှိ Branch ထဲကိုပဲ ယူချင်တဲ့အခြေအနေ ရှိနိုင်ပါတယ်။

ဒီလိုအခြေအနေမှာ **git cherry-pick** ကို အသုံးပြုနိုင်ပါတယ်။

ဥပမာ message.txt ထဲမှာ စာလုံးပေါင်းမှားနေတာရှိပြီး အဲဒီအမှားကို ohmypy Branch ထဲမှာ ပြင်ထားပါတယ်။ ဒါပေမယ့် ohmypy Branch တစ်ခုလုံးကို main ထဲ Merge လုပ်ဖို့ မလိုသေးပါဘူး။

Check the commit history - Commit History ကို ကြည့်ခြင်း

```
git log --all --oneline --graph --decorate
```

ဥပမာ—

```
* 568193c (HEAD -> main) add praise
| * bbce161 (ohmypy) ohmy.sh
| * cbb09c6 fix typo
| * a715138 ohmy.py
|/
* 3a2bd8f add message
```

ဒီမှာ cbb09c6 က fix typo ဆိုတဲ့ Commit ဖြစ်ပြီး ohmypy Branch ထဲမှာ ရှိနေပါတယ်။

Cherry-pick the commit - သတ်မှတ်ထားတဲ့ Commit ကို ရွေးယူခြင်း

main Branch ထဲမှာ ohmypy Branch တစ်ခုလုံးကို Merge မလုပ်ဘဲ fix typo Commit တစ်ခုတည်းကို ယူချင်တဲ့အတွက်—

```
git cherry-pick cbb09c6
```

ကို အသုံးပြုပါတယ်။

အောင်မြင်ရင် Git က main Branch ပေါ်မှာ Commit အသစ်တစ်ခု ဖန်တီးပေးပါလိမ့်မယ်။

```
[main b23d3ee] fix typo
1 file changed, 1 insertion(+), 1 deletion(-)
```

ဒါကြောင့် ohmypy Branch ထဲက fix typo Commit ရဲ့ အပြောင်းအလဲကို main Branch ထဲကို ရွေးချယ်ပြီး ယူလာနိုင်ပါတယ်။

Verify the change - အပြောင်းအလဲကို ပြန်စစ်ခြင်း

Cherry-pick ပြီးနောက် Commit History ကို ပြန်ကြည့်နိုင်ပါတယ်—

```
git log --all --oneline --graph --decorate
```

 ပြီးရင် message.txt ကို ကြည့်ရင်—

```
cat message.txt
git is awesome
```

လို့ ပြောင်းသွားတာကို တွေ့ရပါမယ်။

အဓိက မှတ်ရန်

Merge က Branch တစ်ခုလုံးရဲ့ အပြောင်းအလဲတွေကို ပေါင်းစည်းတာဖြစ်ပြီး၊ **Cherry-pick** ကတော့ လိုအပ်တဲ့ **Commit** တစ်ခုကိုပဲ ရွေးယူပြီး လက်ရှိ Branch ထဲ ထည့်ပေးတာ ဖြစ်ပါတယ်။

```
git cherry-pick <commit>
```

→ သတ်မှတ်ထားတဲ့ Commit ရဲ့ အပြောင်းအလဲကို လက်ရှိ Branch ထဲ ရွေးယူရန်။

Chapter 3 — Local and remote

3.1 Push — Local မှ Remote သို့ ပို့ခြင်း

Clone the remote repo - Remote Repository ကို Clone လုပ်ခြင်း

အခြား Developer တစ်ယောက်က Remote Repository ကို ရယူပြီး ပြင်ဆင်မှုတွေ ပြုလုပ်နိုင်ပါတယ်။

Remote Repository ကို Clone လုပ်ဖို့—

```
git clone https://example.org/remote.git /tmp/alice
```

Clone ပြီးရင် Repository ရှိတဲ့ Directory ထဲကို ဝင်ပါ။

```
cd /tmp/alice
```

Set user name and email - User Name နဲ့ Email သတ်မှတ်ခြင်း

Git Commit တွေအတွက် User Name နဲ့ Email ကို သတ်မှတ်ပါ။

```
git config user.email alice@example.com
```

```
git config user.name "Alice Zakas"
```

Make some changes and commit - File ကို ပြင်ဆင်ပြီး Commit လုပ်ခြင်း

message.txt File ကို ပြင်ဆင်ပါ။

```
echo "Git is awesome!" > message.txt
```

ပြင်ဆင်ထားတဲ့ File ကို Commit လုပ်ပါ။

```
git commit -am "edit from alice"
```

အောင်မြင်ရင်—

```
[main b9714f2] edit from alice
```

1 file changed, 1 insertion(+), 1 deletion(-) လို့ ပြပါလိမ့်မယ်။

Push the committed changes - Commit လုပ်ထားတဲ့ အပြောင်းအလဲကို Remote Repository သို့ ပို့ခြင်း

Local Repository ထဲမှာ Commit လုပ်ထားတဲ့ အပြောင်းအလဲကို Remote Repository ထဲ ပို့ဖို့—

```
git push
```

ကို အသုံးပြုပါတယ်။

ဒီ Command နဲ့ Local Repository ထဲက Commit ကို Remote Repository ဆီ ပို့ပေးနိုင်ပါတယ်။

အဓိက မှတ်ရန်

Local Repository → Remote Repository

Local Repo

↓
`git push`

Remote Repo

git push ဆိုတာ ကိုယ့် Local Repository မှာ Commit လုပ်ပြီးသား အပြောင်းအလဲတွေကို Remote Repository ဆီ ပို့တဲ့

Command ဖြစ်ပါတယ်။

3.2 Pull — Remote မှ Local သို့ ရယူခြင်း

Pull the latest changes from the remote repo

(Remote Repository မှ နောက်ဆုံးအပြောင်းအလဲတွေကို ရယူခြင်း)

အခြား Developer တစ်ယောက်က Remote Repository ထဲကို အပြောင်းအလဲတွေ Push လုပ်ထားပြီး ကိုယ့် Local Repository ထဲမှာ အဲဒီ Commit တွေ မရှိသေးတဲ့အခါ Remote ကနေ အပြောင်းအလဲတွေကို ရယူနိုင်ပါတယ်။

အရင်ဆုံး Local Repository ရဲ့ Commit History ကို ကြည့်ပါ။

```
git log --oneline
```

ဥပမာ—

```
3a2bd8f add message
```

ဒီအချိန်မှာ Alice ရဲ့ Commit က Local Repository ထဲမှာ မရှိသေးပါဘူး။

Run git pull - Remote Repository မှ အပြောင်းအလဲတွေကို Pull လုပ်ခြင်း

Remote Repository ထဲက နောက်ဆုံးအပြောင်းအလဲတွေကို ရယူဖို့—

```
git pull
```

ကို အသုံးပြုပါတယ်။

ဥပမာ—

```
Updating 3a2bd8f..b9714f2
```

```
Fast-forward
```

```
message.txt | 2 +-  
1 file changed, 1 insertion(+), 1 deletion(-)
```

```
lို့ ပြပါတယ်။
```

ဒီမှာ Fast-forward ဖြစ်ပြီး Local Repository ရဲ့ Branch က Remote Repository ရဲ့ နောက်ဆုံး Commit အထိ တိုက်ရိုက် ရွေ့သွားပါတယ်။

Check the updated commit history -အပ်ဒိတ်လုပ်ပြီးတဲ့ Commit History ကို ပြန်ကြည့်ခြင်း

git pull လုပ်ပြီးနောက် Commit History ကို ပြန်ကြည့်နိုင်ပါတယ်။

```
git log --oneline
```

ဥပမာ—

```
b9714f2 edit from alice
```

```
3a2bd8f add message
```

ဒီလိုနဲ့ Remote Repository ထဲမှာ ရှိနေတဲ့ Alice ရဲ့ Commit ကို Local Repository ထဲ ရယူပြီးပါပြီ။

အဓိက မှတ်ရန်

Remote Repository → Local Repository

Remote Repo

↓ git pull

Local Repo

git pull ဆိုတာ Remote Repository ထဲက နောက်ဆုံးအပြောင်းအလဲတွေကို Local Repository ထဲ ရယူတဲ့ Command ဖြစ်ပါတယ်။

3.3 Resolve conflict — Conflict ကို ဖြေရှင်းခြင်း

Resolve a conflict between local and remote changes

(Local နဲ့ Remote အပြောင်းအလဲတွေကြားက Conflict ကို ဖြေရှင်းခြင်း)

တစ်ခါတလေ ကိုယ့် Local Repository မှာ Commit တစ်ခု ပြုလုပ်ထားပြီးသားဖြစ်ပေမယ့်၊ အဲဒီအချိန်မှာ အခြား Developer

က Remote Repository ထဲကို တူညီတဲ့ File ကို ပြင်ပြီး Push လုပ်ထားနိုင်ပါတယ်။

ဒီလိုအခြေအနေမှာ Local နဲ့ Remote အပြောင်းအလဲတွေကို တစ်ပြိုင်နက်တည်း ပေါင်းစည်းလို့မရတဲ့အတွက် **Merge Conflict** ဖြစ်လာနိုင်ပါတယ်။

Pull the remote changes - Remote Repository က အပြောင်းအလဲတွေကို Pull လုပ်ခြင်း

`git pull`

ဒီအချိန်မှာ Git က Conflict ဖြစ်နေကြောင်း ပြပါလိမ့်မယ်။

Auto-merging message.txt

CONFLICT (content): Merge conflict in message.txt

Automatic merge failed; fix conflicts and then commit the result.

ဒါဟာ message.txt ထဲမှာ Local နဲ့ Remote Version နှစ်ခုရဲ့ အပြောင်းအလဲတွေ တစ်နေရာတည်းမှာ ကွဲပြားနေတာကြောင့် ဖြစ်ပါတယ်။

View the conflict - Conflict ဖြစ်နေတဲ့ File ကို ကြည့်ခြင်း

`cat message.txt`

ဥပမာ—

<<<<<< HEAD

git is awesome

=====

Git is awesome!

>>>>>> b9714f2

ဒီမှာ—

<<<<<< HEAD

က ကိုယ့် Local Version ကို ပြပါတယ်။

=====

က Local Version နဲ့ Remote Version ကို ခွဲပြတဲ့ အမှတ်အသား ဖြစ်ပါတယ်။

>>>>>> b9714f2

က Remote Commit Version ကို ပြပါတယ်။

Choose the remote version - Remote Version ကို ရွေးချယ်ခြင်း

PDF ထဲက ဥပမာမှာ Alice ရဲ့ Version ကို ပိုနှစ်သက်တဲ့အတွက် Remote Version ကို ရွေးချယ်ပါတယ်။

`git checkout --theirs -- message.txt`

ကို အသုံးပြုနိုင်ပါတယ်။

ကိုယ့် Local Version ကို ရွေးချယ်ချင်ရင်—

`git checkout --ours -- message.txt`

ကို အသုံးပြုနိုင်ပါတယ်။

Add the resolved file - Conflict ဖြေရှင်းပြီးတဲ့ File ကို Staging Area ထဲ ထည့်ခြင်း

Conflict ကို ဖြေရှင်းပြီးတဲ့နောက်—

```
git add message.txt
```

နဲ့ File ကို Staging Area ထဲ ထည့်ပါ။

Complete the merge - Merge ကို အပြီးသတ် Commit လုပ်ခြင်း

ပြီးရင် Merge ကို အပြီးသတ်ဖို့—

```
git commit -m "merge alice"
```

ကို အသုံးပြုပါတယ်။

အောင်မြင်ရင်—

```
[main cbb6112] merge alice
```

လို့ ပြပါလိမ့်မယ်။

အဓိက မှတ်ရန်

Conflict ဖြစ်တဲ့အခါ အဓိကလုပ်ဆောင်ရမယ့် အဆင့်တွေက—

```
git pull
```

↓

Conflict ဖြစ်

↓

Conflict ဖြစ်နေတဲ့ File ကို စစ်ဆေး

↓

Local / Remote Version ရွေးချယ်

↓

```
git add message.txt
```

↓

```
git commit -m "merge alice"
```

ဒီလိုနဲ့ Local နဲ့ Remote အပြောင်းအလဲတွေကြားက Conflict ကို ဖြေရှင်းပြီး Merge ကို အပြီးသတ်နိုင်ပါတယ်။

3.4 Push branch — Branch ကို Remote သို့ ပို့ခြင်း

Create the local branch - Local Repository မှာ Branch အသစ် ဖန်တီးခြင်း

အရင်ဆုံး ohmypy ဆိုတဲ့ Local Branch အသစ်တစ်ခုကို ဖန်တီးပါမယ်။

```
git branch ohmypy
```

ပြီးရင် ohmypy Branch ကို ပြောင်းသုံးပါမယ်။

```
git switch ohmypy
```

အောင်မြင်ရင်—

Switched to branch 'ohmypy'

လို့ ပြပါလိမ့်မယ်။

Add and commit a file - File ထည့်ပြီး Commit လုပ်ခြင်း

ohmypy Branch ထဲမှာ ohmy.py File အသစ်တစ်ခု ဖန်တီးပါမယ်။

```
echo "print('git is awesome')" > ohmy.py
```

ပြီးရင် File ကို Staging Area ထဲ ထည့်ပြီး Commit လုပ်ပါမယ်။

```
git add ohmy.py
```

```
git commit -m "ohmy.py"
```

အောင်မြင်ရင်—

[ohmypy c64073e] ohmy.py

1 file changed, 1 insertion(+)

create mode 100644 ohmy.py

လို့ ပြပါလိမ့်မယ်။

Push the local branch to the remote - Local Branch ကို Remote Repository သို့ ပို့ခြင်း

Local မှာရှိတဲ့ ohmypy Branch ကို Remote Repository ထဲကို ပို့ဖို့—

```
git push -u origin ohmypy
```

ကို အသုံးပြုပါတယ်။

အောင်မြင်ရင်—

branch 'ohmypy' set up to track 'origin/ohmypy'.

လို့ ပြပါလိမ့်မယ်။

ဒီလိုနဲ့ Local ohmypy Branch နဲ့ Remote origin/ohmypy Branch တို့ကို ချိတ်ဆက်ပေးလိုက်ပါတယ်။

Show local and remote branches - Local နဲ့ Remote Branch တွေကို ကြည့်ခြင်း

Branch အားလုံးကို ကြည့်ဖို့—

`git branch --all`

ကို အသုံးပြုနိုင်ပါတယ်။

ဥပမာ—

```
main
* ohmypy
remotes/origin/main
remotes/origin/ohmypy
```

ဒီမှာ—

- main → Local Branch
- ohmypy → လက်ရှိအသုံးပြုနေတဲ့ Local Branch
- remotes/origin/main → Remote ရဲ့ main Branch
- remotes/origin/ohmypy → Remote ရဲ့ ohmypy Branch

ဖြစ်ပါတယ်။

အဓိက မှတ်ရန်

`git push -u origin ohmypy`

→ Local ohmypy Branch ကို Remote origin ဆီ ပို့ပြီး Remote Branch နဲ့ Tracking ချိတ်ဆက်ရန်။

Local Branch

↓
| git push -u origin ohmypy
↓

Remote Branch
origin/ohmypy

3.5 Fetch branch — Remote Branch ကို ရယူခြင်း

Fetch remote branches - Remote Repository မှ Branch တွေကို ရယူခြင်း

တစ်ခါတလေ Remote Repository မှာ Branch အသစ်တစ်ခု ရှိနေပြီး ကိုယ့် Local Repository မှာတော့ အဲဒီ Branch ကို မရှိသေးနိုင်ပါတယ်။

ဒီလိုအခြေအနေမှာ Remote Repository ရဲ့ Branch အချက်အလက်တွေကို ရယူဖို့—

`git fetch`

ကို အသုံးပြုပါတယ်။

`git fetch` က Remote Repository မှာ ရှိနေတဲ့ နောက်ဆုံး Branch နဲ့ Commit အချက်အလက်တွေကို Local Repository ဆီ ရယူပေးပါတယ်။

Show local branches - Local Branch တွေကို ကြည့်ခြင်း

Fetch လုပ်ပြီးနောက် Local Branch တွေကို ကြည့်ဖို့—

`git branch`

ကို အသုံးပြုနိုင်ပါတယ်။

PDF ထဲက ဥပမာမှာ—

* main

လို့ ပြပါတယ်။

ဒီအချိန်မှာ Remote မှာ ohmypy Branch ရှိပေမယ့် Local မှာတော့ ohmypy Branch ကို မရောက်သေးပါဘူး။

Checkout the remote branch - Remote Branch ကို Local မှာ အသုံးပြုခြင်း

ohmypy Branch ကို Local မှာ Checkout လုပ်ဖို့—

`git switch ohmypy`

ကို အသုံးပြုနိုင်ပါတယ်။

အခြားနည်းလမ်းအနေနဲ့—

`git checkout ohmypy`

ကိုလည်း အသုံးပြုနိုင်ပါတယ်။

အောင်မြင်ရင်—

branch 'ohmypy' set up to track 'origin/ohmypy'.

လို့ ပြပါလိမ့်မယ်။

ဒါဟာ Local ohmypy Branch ကို Remote origin/ohmypy Branch နဲ့ **Tracking** ချိတ်ဆက်ပေးလိုက်တာ ဖြစ်ပါတယ်။

Show branches - Branch တွေကို ပြန်ကြည့်ခြင်း

`git branch`

ကို ပြန် run လုပ်ရင်—

main

* ohmypy

လို့ ပြပါလိမ့်မယ်။

* ohmypy ဖြစ်နေတဲ့အတွက် လက်ရှိအသုံးပြုနေတဲ့ Branch က ohmypy ဖြစ်သွားပါပြီ။

အဓိက မှတ်ရန်

`git fetch`

→ Remote Repository မှာရှိတဲ့ Branch နဲ့ Commit အချက်အလက်တွေကို Local Repository ဆီ ရယူရန်။

`git switch ohmypy`

→ ရယူထားတဲ့ ohmypy Branch ကို Local မှာ အသုံးပြုရန်။

အဓိက ကွာခြားချက်

`git fetch`

↓

Remote ရဲ့ အချက်အလက်ကို ရယူ

`git switch ohmypy`

↓

ohmypy Branch ကို Local မှာ အသုံးပြု

3.6 Tags – Tag များ အသုံးပြုခြင်း

Create a tag for the latest commit - နောက်ဆုံး Commit အတွက် Tag တစ်ခု ဖန်တီးခြင်း

Repository ရဲ့ လက်ရှိနောက်ဆုံး Commit ကို အမည်တစ်ခု သတ်မှတ်ပေးဖို့ git tag ကို အသုံးပြုနိုင်ပါတယ်။

```
git tag 0.1.0 HEAD
```

ဒီမှာ 0.1.0 က Tag အမည်ဖြစ်ပြီး HEAD က လက်ရှိနောက်ဆုံး Commit ကို ရည်ညွှန်းပါတယ်။

Create a tag for an older commit - အရင် Commit တစ်ခုအတွက် Tag ဖန်တီးခြင်း

လက်ရှိ Commit မဟုတ်ဘဲ အရင် Commit တစ်ခုကို Tag သတ်မှတ်ချင်ရင် HEAD~n ကို အသုံးပြုနိုင်ပါတယ်။

```
git tag 0.1.0-alpha HEAD~1
```

ဒီမှာ HEAD~1 က လက်ရှိ Commit ရဲ့ တစ်ဆင့်အရင် Commit ကို ဆိုလိုပါတယ်။

Commit hash ကို တိုက်ရိုက်အသုံးပြုပြီးလည်း Tag ဖန်တီးနိုင်ပါတယ်။

Show tags - Tag တွေကို ကြည့်ခြင်း

Repository ထဲမှာရှိတဲ့ Tag တွေကို ကြည့်ဖို့—

```
git tag -l
```

ကို အသုံးပြုပါတယ်။

ဥပမာ—

0.1.0

0.1.0-alpha

လို့ ပြပါလိမ့်မယ်။

Show compact log with tags - Tag တွေနဲ့အတူ Commit History ကို တိုတောင်းတဲ့ပုံစံနဲ့ ကြည့်ခြင်း

```
git log --decorate --oneline
```

ဥပမာ—

ecdeb79 (HEAD -> main, tag: 0.1.0, origin/main) edit message

3a2bd8f (tag: 0.1.0-alpha) add message

ဒီလိုကြည့်ရင် ဘယ် Commit မှာ ဘယ် Tag ရှိနေတယ်ဆိုတာကို တစ်ခါတည်း မြင်နိုင်ပါတယ်။

Delete a tag - Tag ကို ဖျက်ခြင်း

Tag တစ်ခုကို ဖျက်ချင်ရင်—

```
git tag -d 0.1.0-alpha
```

ကို အသုံးပြုပါတယ်။

အောင်မြင်ရင်—

Deleted tag '0.1.0-alpha' (was 3a2bd8f)

လို့ ပြပါလိမ့်မယ်။

Push tags to the remote - Tag တွေကို Remote Repository သို့ ပို့ခြင်း

Local Repository မှာ ဖန်တီးထားတဲ့ Tag တွေကို Remote Repository သို့ ပို့ဖို့—

```
git push --tags
```

ကို အသုံးပြုပါတယ်။

အဓိက မှတ်ရန်

```
git tag 0.1.0 HEAD
```

→ လက်ရှိ Commit ကို Tag သတ်မှတ်ရန်

```
git tag -l
```

→ Tag တွေကို ကြည့်ရန်

```
git tag -d 0.1.0-alpha
```

→ Tag ဖျက်ရန်

```
git push --tags
```

→ Tag တွေကို Remote သို့ ပို့ရန်

Tag ကို Commit တစ်ခုကို အလွယ်တကူ ပြန်မှတ်မိနိုင်ဖို့ **အမည်တစ်ခု သတ်မှတ်ပေးတဲ့နည်းလမ်း** အဖြစ် အသုံးပြုနိုင်ပါတယ်။

4.1 Amend commit – Commit ကို ပြန်လည်ပြင်ဆင်ခြင်း

Edit a file and commit - File ကို ပြင်ဆင်ပြီး Commit လုပ်ခြင်း

အရင်ဆုံး message.txt ကို ပြင်ဆင်ပြီး Commit လုပ်ပါမယ်။

```
echo "git is awesome" > message.txt
```

```
git commit -am "edit nessage"
```

ဒီနေရာမှာ Commit Message ကို edit nessage လို့ စာလုံးပေါင်းမှားပြီး Commit လုပ်မိပါတယ်။

```
[main c0206a0] edit nessage
```

```
1 file changed, 1 insertion(+), 1 deletion(-)
```

Show commits - Commit မှတ်တမ်းကို ကြည့်ခြင်း

```
git log --oneline
```

ရလဒ်က—

```
c0206a0 edit nessage
```

```
3a2bd8f add message
```

ဖြစ်ပါတယ်။

Change the last commit message - နောက်ဆုံး Commit ရဲ့ Message ကို ပြန်ပြင်ခြင်း

Commit Message မှားနေတဲ့အတွက် နောက်ဆုံး Commit ကို ပြန်ပြင်ဖို့—

```
git commit --amend -m "edit message"
```

ကို အသုံးပြုပါတယ်။

အောင်မြင်ရင်—

```
[main ecdeb79] edit message
```

```
Date: Thu Mar 14 15:00:00 2024 +0000
```

```
1 file changed, 1 insertion(+), 1 deletion(-)
```

လို့ ပြပါလိမ့်မယ်။

Git replaces the last commit - Git က နောက်ဆုံး Commit ကို အစားထိုးပေးခြင်း

git commit --amend လုပ်ပြီးတဲ့အခါ Git က အရင်နောက်ဆုံး Commit ကို အသစ်ပြင်ထားတဲ့ Commit နဲ့ အစားထိုးပေးပါ

တယ်။ ဒါကြောင့် Commit History မှာ c0206a0 edit nessage အစား ecdeb79 edit message ဖြစ်သွားပါတယ်။

နောက်ဆုံး Commit တစ်ခုမဟုတ်ဘဲ နောက်ဆုံး Commit n ခုထဲက တစ်ခုကို ပြင်ချင်ရင်—

```
git rebase -i HEAD~n
```

ကို အသုံးပြုနိုင်ပါတယ်။

မှတ်ချက်

Amend only works if the commit has not yet been pushed to the remote repo!

ဆိုတာပါ။

အဓိပ္ပါယ်က Remote Repository ကို မ Push ရသေးတဲ့ Commit တွေအတွက်သာ git commit --amend ကို အသုံးပြု

သင့်ပါတယ်။

အဓိက မှတ်ရန်

```
git commit --amend -m "edit message"
```

→ နောက်ဆုံး Commit ကို ပြန်ပြင်ပြီး Commit Message ကို ပြောင်းရန်။

မူရင်း

```
c0206a0 edit message
```

↓ git commit --amend

အသစ်

```
ecdeb79 edit message
```

Amend = နောက်ဆုံး Commit ကို ပြန်ပြင်ပြီး အစားထိုးခြင်း ဖြစ်ပါတယ်။

4.2 Undo uncommitted changes —

(Commit မလုပ်ရသေးသော အပြောင်းအလဲများကို ပြန်ပယ်ဖျက်ခြင်း)

Edit the previously committed file - အရင် Commit လုပ်ထားတဲ့ File ကို ပြင်ဆင်ခြင်း

အရင်က Commit လုပ်ထားတဲ့ message.txt ကို ပြင်ဆင်ပြီး Staging Area ထဲ ထည့်ပါမယ်။

```
echo "git is awesome" > message.txt  
git add message.txt
```

Show the working tree status - Working Tree ရဲ့ လက်ရှိအခြေအနေကို ကြည့်ခြင်း

```
git status
```

ရလဒ်မှာ—

On branch main

Changes to be committed:

modified: message.txt

လို့ ပြပါလိမ့်မယ်။

ဒီအချိန်မှာ message.txt ရဲ့ အပြောင်းအလဲဟာ **Staging Area** ထဲ ရောက်နေပါတယ်။

Remove the changes from the staging area - Staging Area ထဲက အပြောင်းအလဲကို ပြန်ဖယ်ခြင်း

Staging Area ထဲက message.txt ကို ပြန်ဖယ်ချင်ရင်—

```
git restore --staged message.txt
```

ကို အသုံးပြုပါတယ်။

ဒီအချိန်မှာ File က **Staging Area** ထဲမှာ မရှိတော့ပေမယ့် **Local File** အနေနဲ့တော့ ပြင်ဆင်ဆဲ ဖြစ်ပါတယ်။

Discard the changes altogether - အပြောင်းအလဲအားလုံးကို လုံးဝပြန်ပယ်ဖျက်ခြင်း

အဲဒီ Local File ရဲ့ ပြင်ဆင်မှုကိုပါ လုံးဝမလိုချင်တော့ဘူးဆိုရင်—

```
git restore message.txt
```

ကို အသုံးပြုနိုင်ပါတယ်။

အဟောင်း Git Syntax အနေနဲ့—

```
git checkout message.txt
```

ကိုလည်း အသုံးပြုနိုင်ပါတယ်။

ပြီးရင်—

```
cat message.txt
```

နဲ့ File ကို ပြန်ကြည့်ရင် မူရင်းအကြောင်းအရာ—

```
git is awesom
```

ကို ပြန်ရရှိပါလိမ့်မယ်။

အဓိက မှတ်ရန်

```
git add message.txt
```

↓

Staging Area ထဲ ရောက်

↓

```
git restore --staged message.txt
```

↓

Staging Area ကနေ ဖယ်

↓

```
git restore message.txt
```

↓

ပြင်ထားတာကို လုံးဝပယ်ဖျက်

git restore --staged → Staging Area ကနေ ဖယ်ခြင်း

git restore → Working Directory ထဲက ပြင်ထားတဲ့အပြောင်းအလဲကိုပါ ပြန်ပယ်ဖျက်ခြင်း

4.3 Undo local commit — Local Commit ကို ပြန်ပယ်ဖျက်ခြင်း

Undo the last local commit - နောက်ဆုံး Local Commit ကို ပြန်ပယ်ဖျက်ခြင်း

တစ်ခါတလေ နောက်ဆုံးလုပ်ထားတဲ့ Commit ကို မလိုချင်တော့ဘဲ ပြန်ပယ်ဖျက်ချင်တဲ့အခြေအနေ ရှိနိုင်ပါတယ်။

ဒီအပိုင်းမှာ **Remote Repository** ကို မ **Push** ရသေးတဲ့ **Local Commit** ကို ပြန်ပယ်ဖျက်တဲ့နည်းကို လေ့လာပါမယ်။

Show commits - Commit မှတ်တမ်းကို ကြည့်ခြင်း

```
git log --oneline
```

ဥပမာ—

```
ecdeb79 edit message
```

```
3a2bd8f add message
```

ဒီမှာ ecdeb79 က နောက်ဆုံး Commit ဖြစ်ပါတယ်။

Undo the last commit - နောက်ဆုံး Commit ကို ပြန်ပယ်ဖျက်ခြင်း

နောက်ဆုံး Commit ကို ပြန်ပယ်ဖျက်ဖို့—

```
git reset --soft HEAD~
```

ကို အသုံးပြုပါတယ်။

ပြီးရင် Commit History ကို ပြန်ကြည့်ရင်—

```
git log --oneline
```

3a2bd8f add message

လို့ ပြပြီး ecdeb79 edit message Commit က History ထဲကနေ ပျောက်သွားပါလိမ့်မယ်။

The changes are still staged

(Commit ပယ်ဖျက်သွားပေမယ့် အပြောင်းအလဲတွေက Staging Area ထဲမှာ ရှိနေခြင်း)

--soft ကို အသုံးပြုထားတဲ့အတွက် Commit ကိုပဲ ပြန်ပယ်ဖျက်ပြီး အဲဒီ Commit ထဲက အပြောင်းအလဲတွေကို Staging Area ထဲမှာ ဆက်ထားပေးပါတယ်။

```
git status
```

ရလဒ်မှာ—

On branch main

Changes to be committed:

modified: message.txt

လို့ ပြပါလိမ့်မယ်။

Remove both the commit and local changes - Commit နဲ့ Local အပြောင်းအလဲ နှစ်ခုလုံးကို ပယ်ဖျက်ခြင်း

Commit ကိုသာမက အဲဒီ Commit ထဲက Local Changes တွေကိုပါ လုံးဝပယ်ဖျက်ချင်ရင် --soft အစား --hard ကို အသုံးပြုပါတယ်။

```
git reset --hard HEAD~
```

ပြီးရင်—

```
git status
```

ရလဒ်မှာ—

HEAD is now at 3a2bd8f add message

On branch main

nothing to commit, working tree clean

လို့ ပြပါလိမ့်မယ်။

မှတ်ချက်

git reset ကို Remote Repository သို့ Push လုပ်ပြီးသား Commit အတွက် မသုံးသင့်ပါဘူး။

ဒီအပိုင်းမှာ လေ့လာထားတာက Local မှာပဲ ရှိနေသေးပြီး Remote ကို မ Push ရသေးတဲ့ Commit ကို ပြန်ပယ်ဖျက်တဲ့ နည်း ဖြစ်ပါတယ်။

4.4 Undo remote commit — Remote သို့ Push လုပ်ပြီးသား Commit ကို ပြန်ပယ်ဖျက်ခြင်း

Undo a commit that has already been pushed

(Remote Repository သို့ Push လုပ်ပြီးသား Commit ကို ပြန်ပယ်ဖျက်ခြင်း)

တစ်ခါတလေ နောက်ဆုံးလုပ်ထားတဲ့ Commit ကို ပြန်ပယ်ဖျက်ချင်ပေမယ့် အဲဒီ Commit က Remote Repository သို့

Push လုပ်ပြီးသား ဖြစ်နေနိုင်ပါတယ်။

ဒီလိုအခြေအနေမှာ Local Commit ကို git reset နဲ့ ဖျက်ပစ်လို့ မရတော့ပါဘူး။

Show commits - Commit မှတ်တမ်းကို ကြည့်ခြင်း

```
git log --oneline
```

ဥပမာ—

ecdeb79 edit message

3a2bd8f add message

ဒီမှာ ecdeb79 က Remote Repository သို့ Push လုပ်ပြီးသား နောက်ဆုံး Commit ဖြစ်ပါတယ်။

Revert the last commit - နောက်ဆုံး Commit ကို Revert လုပ်ခြင်း

Remote ကို Push လုပ်ပြီးသားဖြစ်တဲ့အတွက် Commit ကို History ထဲကနေ တိုက်ရိုက်ဖျက်မယ့်အစား **Undo လုပ်ပေးတဲ့**

Commit အသစ်တစ်ခု ဖန်တီးရပါတယ်။

```
git revert HEAD --no-edit
```

အောင်မြင်ရင်—

[main 9ffa044] Revert "edit message"

Date: Thu Mar 14 15:00:00 2024 +0000

1 file changed, 1 insertion(+), 1 deletion(-)

လို့ ပြပါလိမ့်မယ်။

Check the commit history - Commit History ကို ပြန်ကြည့်ခြင်း

```
git log --oneline
```

ရလဒ်က—

9ffa044 Revert "edit message"

ecdeb79 edit message

3a2bd8f add message

ဖြစ်လာပါမယ်။

သတိပြုရမှာက **ecdeb79 Commit ကို History ထဲကနေ ဖျက်လိုက်တာ မဟုတ်ပါဘူး။** အဲဒီ Commit ရဲ့ အပြောင်းအလဲကို

ပြန်ပယ်ဖျက်တဲ့ 9ffa044 ဆိုတဲ့ **Revert Commit အသစ်** တစ်ခုကို ဖန်တီးလိုက်တာ ဖြစ်ပါတယ်။

Push the “undo” commit - Undo Commit ကို Remote Repository သို့ ပြန်ပို့ခြင်း

Revert Commit အသစ်ကို Remote Repository ထဲ ပို့ဖို့—

```
git push
```

ကို အသုံးပြုပါတယ်။

HEAD~n or commit hash

နောက်ဆုံး Commit မဟုတ်ဘဲ **နောက်ဆုံးကနေ n ဆင့်အရင်က Commit** ကို Revert လုပ်ချင်ရင်—

`git revert HEAD~n`

ကို အသုံးပြုနိုင်ပါတယ်။

သတ်မှတ်ထားတဲ့ Commit ရဲ့ Hash ကို တိုက်ရိုက်သုံးပြီးလည်း—

`git revert <commit-hash>`

လုပ်နိုင်ပါတယ်။

အဓိက မှတ်ရန်

Local Commit ဖြစ်ရင်—

`git reset --hard HEAD~`

→ Commit ကို History ကနေ ပြန်ဖယ်နိုင်ပါတယ်။

Remote သို့ Push လုပ်ပြီးသား Commit ဖြစ်ရင်—

`git revert HEAD`

→ မူရင်း Commit ကို မဖျက်ဘဲ **Undo Commit အသစ်တစ်ခု** ဖန်တီးပါတယ်။

ecdeb79 edit message

↓

git revert HEAD

↓

9ffa044 Revert "edit message"

↓

git push

အလွယ်မှတ်ရန် — Push လုပ်ပြီးသား Commit ကို reset နဲ့ မဖျက်ဘဲ revert နဲ့ Undo လုပ်ပါတယ်။

4.5 Rewind history — Commit History ကို နောက်သို့ ပြန်လှည့်ခြင်း

Show commits - Commit မှတ်တမ်းကို ကြည့်ခြင်း

အရင်ဆုံး Repository ရဲ့ Commit History ကို ကြည့်ပါမယ်။

`git log --oneline --graph`

ဥပမာ—

* 7d5ac4f Merge branch 'ohmypy'

|\

| * a715138 ohmy.py

* | ecdeb79 edit message

|/

* 3a2bd8f add message

ဒီလိုနဲ့ Repository ရဲ့ Commit တွေ ဘယ်လို ဆက်စပ်နေတယ်ဆိုတာကို Graph ပုံစံနဲ့ မြင်နိုင်ပါတယ်။

Show all repository states - Repository ရဲ့ ယခင်အခြေအနေတွေကို ကြည့်ခြင်း

Git မှာ reflog ကို အသုံးပြုပြီး HEAD ရဲ့ အရင်အခြေအနေတွေကို နောက်ဆုံးဖြစ်ခဲ့တာကနေ အရင်ဖြစ်ခဲ့တာအထိ ကြည့်နိုင်ပါတယ်။

`git reflog`

ဥပမာ—

```
7d5ac4f HEAD@{0}: merge ohmypy
ecdeb79 HEAD@{1}: commit: edit message
3a2bd8f HEAD@{2}: checkout: moving from ohmypy to main
a715138 HEAD@{3}: commit: ohmy.py
3a2bd8f HEAD@{4}: checkout: moving from main to ohmypy
3a2bd8f HEAD@{5}: commit (initial): add message
```

ဒီမှာ HEAD@{0}, HEAD@{1}, HEAD@{2} စသဖြင့် HEAD ရဲ့ ယခင်အခြေအနေတွေ ကို ရည်ညွှန်းထားပါတယ်။

Go back to an earlier state - ယခင် Repository အခြေအနေတစ်ခုကို ပြန်သွားခြင်း

ဥပမာ HEAD@{3} အခြေအနေကို ပြန်သွားချင်တယ်ဆိုရင်—

`git reset --hard HEAD@{3}`

ကို အသုံးပြုနိုင်ပါတယ်။

အောင်မြင်ရင်—

HEAD is now at a715138 ohmy.py

လို့ ပြပါလိမ့်မယ်။

Check the history again - ပြန်လှည့်ပြီးနောက် History ကို ပြန်ကြည့်ခြင်း

`git log --oneline --graph`

ရလဒ်မှာ—

```
* a715138 ohmy.py
* 3a2bd8f add message
```

လို ဖြစ်လာပြီး Repository က HEAD@{3} အချိန်က အခြေအနေကို ပြန်ရောက်သွားပါတယ်။

အဓိက မှတ်ရန်

`git reflog`

→ HEAD ရဲ့ ယခင်အခြေအနေတွေကို ကြည့်ရန်။

`git reset --hard HEAD@{3}`

→ သတ်မှတ်ထားတဲ့ ယခင်အခြေအနေကို Repository နဲ့ Working Tree နှစ်ခုလုံး ပြန်ထားရန်။

လက်ရှိ

↓
HEAD@{0}

↓
HEAD@{1}

↓
HEAD@{2}

↓
HEAD@{3} ← ဒီနေရာကို ပြန်သွား

Rewind history ဆိုတာ reflog မှာ မှတ်တမ်းတင်ထားတဲ့ ယခင် HEAD အခြေအနေတစ်ခုကို ရွေးပြီး Repository ကို အဲဒီ အခြေအနေဆီ ပြန်လှည့်ပေးခြင်း ဖြစ်ပါတယ်။

4.6 Stash changes – အပြောင်းအလဲများကို ယာယီသိမ်းဆည်းခြင်း

Make some changes - File ကို ပြင်ဆင်ခြင်း

အရင် Commit လုပ်ထားတဲ့ message.txt ကို ပြင်ဆင်ပါမယ်။

```
echo "git is awesome" > message.txt  
git add message.txt
```

ဒီအချိန်မှာ File ရဲ့ အပြောင်းအလဲဟာ Staging Area ထဲမှာ ရှိနေပါတယ်။

Stash the local changes - Local အပြောင်းအလဲတွေကို ယာယီသိမ်းဆည်းခြင်း

အခု Branch ပြောင်းချင်ပေမယ့် လက်ရှိပြင်ထားတဲ့ Changes တွေကို Commit မလုပ်ချင်သေးဘူးဆိုရင်—

```
git stash
```

ကို အသုံးပြုနိုင်ပါတယ်။

အောင်မြင်ရင်—

Saved working directory and index state WIP on main:

3a2bd8f add message

လို့ ပြပါလိမ့်မယ်။

git stash က Working Directory နဲ့ Staging Area ထဲက အပြောင်းအလဲတွေကို ယာယီသိမ်းဆည်းထားပြီး Working Tree

ကို သန့်ရှင်းတဲ့အခြေအနေ ဖြစ်အောင် လုပ်ပေးပါတယ်။

Stash multiple changes - အပြောင်းအလဲများကို အကြိမ်ကြိမ် Stash လုပ်ခြင်း

Stash က **Stack** တစ်ခုလို အလုပ်လုပ်တာကြောင့် အပြောင်းအလဲတွေကို အကြိမ်များစွာ ယာယီသိမ်းဆည်းနိုင်ပါတယ်။

ဥပမာ နောက်ထပ် Change တစ်ခုလုပ်ပြီး—

```
echo "Git is awesome!" > message.txt  
git stash
```

ထပ်လုပ်နိုင်ပါတယ်။

ဒီလိုနဲ့ Stash နှစ်ခု ဖြစ်လာပါမယ်။

Show stash contents - Stash ထဲမှာ သိမ်းထားတဲ့ အပြောင်းအလဲတွေကို ကြည့်ခြင်း

Stash တွေကို ကြည့်ဖို့—

```
git stash list
```

ကို အသုံးပြုပါတယ်။

ဥပမာ—

stash@{0}: WIP on main: 3a2bd8f add message

stash@{1}: WIP on main: 3a2bd8f add message

ဒီမှာ stash@{0} က နောက်ဆုံးသိမ်းထားတဲ့ Stash ဖြစ်ပြီး stash@{1} က အဲဒီမတိုင်ခင် သိမ်းထားတဲ့ Stash ဖြစ်ပါတယ်။

Switch branches while changes are stashed - Stash လုပ်ထားပြီး Branch ပြောင်းခြင်း

Changes တွေကို Stash လုပ်ပြီးတဲ့အခါ အခြား Branch ကို လွတ်လပ်စွာ ပြောင်းပြီး အလုပ်လုပ်နိုင်ပါတယ်။

`git switch <another-branch>`

အလုပ်ပြီးတဲ့အခါ main Branch ကို ပြန်ပြောင်းပြီး Stash ထဲက Changes တွေကို ပြန်ယူနိုင်ပါတယ်။

Re-apply the latest stash - နောက်ဆုံး Stash ကို ပြန်အသုံးပြုခြင်း

နောက်ဆုံးသိမ်းထားတဲ့ Stash ကို ပြန်ယူဖို့—

`git stash pop`

ကို အသုံးပြုပါတယ်။

pop က Stash Stack ထဲက Changes တွေကို နောက်ဆုံးဝင်တာ အရင်ထွက်တဲ့ (LIFO — Last In, First Out) ပုံစံနဲ့ ပြန်ယူပေးပါတယ်။

အဓိက မှတ်ရန်

Working Changes



`git stash`



Stash Stack



Branch ပြောင်း / အခြားအလုပ်လုပ်



`git stash pop`



Changes ပြန်ရ

`git stash`

→ လက်ရှိအပြောင်းအလဲတွေကို ယာယီသိမ်းရန်

`git stash list`

→ သိမ်းထားတဲ့ Stash တွေကို ကြည့်ရန်

`git stash pop`

→ နောက်ဆုံး Stash ကို ပြန်အသုံးပြုရန်

Stash = Commit မလုပ်ချင်သေးတဲ့ အပြောင်းအလဲတွေကို ယာယီသိမ်းထားပြီး အခြားအလုပ်ကို ဆက်လုပ်နိုင်အောင်

အသုံးပြုတဲ့ နည်းလမ်း ဖြစ်ပါတယ်။

5.1 Log summary – Commit မှတ်တမ်း အကျဉ်းချုပ်

Show a summary of commits - Commit မှတ်တမ်းတွေကို အကျဉ်းချုပ်ကြည့်ခြင်း

တစ်ခါတလေ Repository ထဲမှာ Commit တွေ အများကြီးရှိတဲ့အတွက် git log နဲ့ တစ်ခုချင်းစီကြည့်တာထက် ဘယ်သူက

Commit ဘယ်နှခုလုပ်ထားလဲ ဆိုတာကို အကျဉ်းချုပ်ကြည့်ချင်နိုင်ပါတယ်။

ဒီလိုအခြေအနေမှာ

`git shortlog` ကို အသုံးပြုနိုင်ပါတယ်။

PDF ထဲက ဥပမာအရ v1.0 Release ကတည်းက **Contributor 3** ယောက်က **Commit 6** ခု ပြုလုပ်ထားပါတယ်။

Group commits by contributors - Contributor အလိုက် Commit တွေကို အုပ်စုခွဲကြည့်ခြင်း

`git shortlog v1.0`

ဒီ Command က v1.0 Tag ကနေ စပြီး Commit တွေကို Contributor အလိုက် အုပ်စုခွဲပြီး ပြပေးပါတယ်။

Customize the log format - Commit Log ရဲ့ ဖော်ပြပုံကို စိတ်ကြိုက်သတ်မှတ်ခြင်း

PDF ထဲမှာ `--pretty=format:` ကို အသုံးပြုပြီး Commit Log မှာ ပြချင်တဲ့ အချက်အလက်တွေကို စိတ်ကြိုက်သတ်မှတ်ထားပါတယ်။

`git log --pretty=format:'%h %an %s %d'`

ဒီမှာ—

- `%h` → **Commit Hash** ကို အတိုကောက်ပြခြင်း
- `%an` → **Author Name** ကို ပြခြင်း
- `%s` → **Commit Message** ကို ပြခြင်း
- `%d` → **Decoration** — Branch Name, Tag စတာတွေကို ပြခြင်း

ဖြစ်ပါတယ်။

ဥပမာ—

7611979 bob ohmy.lua (HEAD -> main, origin/main)

ef4f23e bob ohmy.sh

3d8f700 bob ohmy.py

c61962c alice no debates

2ab82f6 alice go is great

ecdeb79 sandbox edit message

3a2bd8f sandbox add message (tag: v1.0)

ဒီလိုနဲ့ Commit တစ်ခုချင်းစီရဲ့ **Hash + Author + Message + Branch/Tag** ကို တစ်ကြောင်းတည်းနဲ့ ကြည့်နိုင်ပါတယ်။

Useful git shortlog options - git shortlog ရဲ့ အသုံးဝင်တဲ့ Options များ

PDF မှာ အသုံးဝင်တဲ့ Option နှစ်ခုကို ထပ်ဖော်ပြထားပါတယ်—

`git shortlog -n`

→ Contributor တစ်ယောက်ချင်းစီရဲ့ **Commit အရေအတွက်ကို အများဆုံးကနေ အနည်းဆုံးအထိ** စီပေးပါတယ်။

`git shortlog -s`

→ Commit Message တွေကို မပြဘဲ **Commit အရေအတွက်နဲ့ Contributor Name** ကိုပဲ အကျဉ်းချုပ်ပြပါတယ်။

အဓိက မှတ်ရန်

`git shortlog v1.0`

→ v1.0 ကနေ စပြီး Contributor အလိုက် Commit တွေကို အကျဉ်းချုပ်ကြည့်ရန်။

`git log --pretty=format:'%h %an %s %d'`

→ Commit Log ကို **Hash + Author + Message + Decoration** ပုံစံနဲ့ ပြရန်။

%h → Hash

%an → Author

%s → Message

%d → Branch / Tag

Log summary = Commit History အားလုံးကို အသေးစိတ်ဖတ်မယ့်အား Contributor နဲ့ Commit အချက်အလက်တွေကို အကျဉ်းချုပ်ကြည့်ခြင်း ဖြစ်ပါတယ်။

5.2 Worktree — Worktree များ အသုံးပြုခြင်း

Work on multiple branches at the same time - Branch များကို တစ်ချိန်တည်းမှာ အသုံးပြုခြင်း
တစ်ခါတလေ ohmypy Branch မှာ အရေးကြီးတဲ့ အလုပ်တစ်ခု လုပ်နေတုန်း main Branch မှာ အရေးပေါ် Bug တစ်ခုကို ပြင်ဖို့ လိုလာနိုင်ပါတယ်။

ဒီအခြေအနေမှာ လက်ရှိအလုပ်ကို `git stash` လုပ်ပြီး Branch ပြောင်းနိုင်ပါတယ်။

ဒါမှမဟုတ် `git worktree` ကို အသုံးပြုပြီး Branch နှစ်ခုကို တစ်ချိန်တည်းမှာ Directory သီးခြားတွေနဲ့ အသုံးပြုနိုင်ပါတယ်။

Create a worktree - Worktree အသစ် ဖန်တီးခြင်း

main Branch ကို /tmp/hotfix Directory ထဲမှာ Checkout လုပ်ဖို့—

`git worktree add -b hotfix /tmp/hotfix main`

ကို အသုံးပြုပါတယ်။

ဒီ Command က—

- hotfix ဆိုတဲ့ Branch အသစ် ဖန်တီးခြင်း
- /tmp/hotfix Directory ထဲမှာ Worktree ဖန်တီးခြင်း
- main Branch ကို အခြေခံအသုံးပြုခြင်း

တို့ကို တစ်ခါတည်း လုပ်ပေးပါတယ်။

အောင်မြင်ရင်—

HEAD is now at 3a2bd8f add message

လို့ ပြပါလိမ့်မယ်။

Work inside the new worktree - Worktree အသစ်ထဲမှာ အလုပ်လုပ်ခြင်း

Worktree Directory ထဲကို ဝင်ပါ။

```
cd /tmp/hotfix
```

ပြီးရင် message.txt ကို ပြင်ဆင်ပါ။

```
echo "git is awesome" > message.txt
```

ပြင်ဆင်ပြီး Commit လုပ်ပါ။

```
git commit -am "fix typo"
```

အောင်မြင်ရင်—

```
[hotfix c3485cd] fix typo
```

```
1 file changed, 1 insertion(+), 1 deletion(-)
```

လို့ ပြပါလိမ့်မယ်။

Push the hotfix - Hotfix ကို Remote Repository သို့ ပို့ခြင်း

Hotfix ကို Remote ရဲ့ main Branch သို့ Push လုပ်ဖို့—

```
git push --set-upstream origin main
```

ကို အသုံးပြုပါတယ်။

အောင်မြင်ရင်—

```
branch 'main' set up to track 'origin/main'.
```

လို့ ပြပါလိမ့်မယ်။

Return to the original worktree - မူလ Worktree ကို ပြန်သွားခြင်း

Hotfix ကို ပြင်ပြီးသွားတဲ့အခါ မူလ Repository Directory /tmp/repo ကို ပြန်သွားပြီး ohmypy Branch မှာ လုပ်နေတဲ့ အလုပ်ကို ဆက်လုပ်နိုင်ပါတယ်။

```
/tmp/repo
└── ohmypy Branch
    ↓
    မူလအလုပ် ဆက်လုပ်
```

```
/tmp/hotfix
└── hotfix Branch
    ↓
    Bug ပြင် + Commit
```

အဓိက မှတ်ရန်

```
git worktree add -b hotfix /tmp/hotfix main
```

→ main Branch ကို အခြေခံပြီး /tmp/hotfix မှာ hotfix Worktree အသစ် ဖန်တီးရန်။

Worktree ရဲ့ အဓိကအကျိုးကျေးဇူးက လက်ရှိ Branch မှာ လုပ်နေတဲ့ အလုပ်ကို မရပ်ဘဲ အခြား Branch ကို Directory သီးခြားတစ်ခုမှာ တစ်ပြိုင်နက်တည်း အသုံးပြုနိုင်ခြင်း ဖြစ်ပါတယ်။

5.3 Bisect – Bug ဖြစ်စေသော Commit ကို ရှာဖွေခြင်း

Find the commit that introduced a bug - Bug ကို စတင်ဖြစ်စေခဲ့တဲ့ Commit ကို ရှာဖွေခြင်း

Repository ထဲမှာ Commit တွေ အများကြီးရှိတဲ့အခါ ဘယ် Commit က Bug ကို စတင်ဖြစ်စေခဲ့တာလဲ ဆိုတာကို ရှာဖွေရတာ ခက်ခဲနိုင်ပါတယ်။

ဒီလိုအခြေအနေမှာ Git ရဲ့ **Bisect Algorithm** ကို အသုံးပြုပြီး Bug ဖြစ်စေတဲ့ Commit ကို အဆင့်ဆင့် ရှာဖွေနိုင်ပါတယ်။

Show the commits - Commit History ကို ကြည့်ခြင်း

PDF ထဲမှာ Commit ၅ ခုရှိပါတယ်။

```
git log --oneline
```

ဥပမာ—

```
2f568eb main.sh
31ed915 main.sh
f8b2baf main.sh
5e0cf35 main.sh
8f0f1e4 main.sh
51c34ff test.sh
```

ပြီးရင် Test ကို Run လုပ်ကြည့်ပါတယ်။

```
sh test.sh
```

ရလဒ်က—

```
FAIL (exit status 1)
```

ဖြစ်ပါတယ်။

Start git bisect - Bisect လုပ်ငန်းစဉ်ကို စတင်ခြင်း

```
git bisect start
```

ရလဒ်—

```
status: waiting for both good and bad commits
```

လို့ ပြပါလိမ့်မယ်။

Mark the bad and good commits -

(Bug ဖြစ်နေတဲ့ Commit နဲ့ Bug မဖြစ်သေးတဲ့ Commit ကို သတ်မှတ်ခြင်း)

လက်ရှိ Commit က Bug ဖြစ်နေတဲ့အတွက် **Bad Commit** အဖြစ် သတ်မှတ်ပါမယ်။

```
git bisect bad HEAD
```

ပြီးရင် ပထမဆုံး main.sh Commit က Bug မဖြစ်သေးဘူးလို့ ယုံကြည်ထားတဲ့အတွက် **Good Commit** အဖြစ် သတ်မှတ်ပါမယ်။

```
git bisect good HEAD~4
```

Git က Good နဲ့ Bad ကြားက အလယ် Commit တစ်ခုကို အလိုအလျောက် Checkout လုပ်ပေးပါလိမ့်မယ်။

Test the selected commit - ရွေးချယ်ထားတဲ့ Commit ကို Test လုပ်ခြင်း

Git က Checkout လုပ်ပေးတဲ့ Commit မှာ Test ကို Run လုပ်ပါ။

```
sh test.sh
```

PDF ထဲက ပထမစမ်းသပ်မှုမှာ—

PASS

ဖြစ်တဲ့အတွက် ဒီ Commit ကို **Good** လို့ သတ်မှတ်ပါတယ်။

```
git bisect good
```

ပြီးရင် Git က နောက်ထပ် စစ်ဆေးရမယ့် Commit ကို အလိုအလျောက် ရွေးပေးပါလိမ့်မယ်။

Test again - ထပ်မံ Test လုပ်ခြင်း

နောက်ထပ် Git ရွေးပေးတဲ့ Commit မှာ—

```
sh test.sh
```

လုပ်တဲ့အခါ—

FAIL (exit status 1)

ဖြစ်ပါတယ်။

ဒါကြောင့် ဒီ Commit ကို **Bad** လို့ သတ်မှတ်ပါတယ်။

```
git bisect bad
```

ဒီ Commit ရဲ့ Hash က—

31ed915

ဖြစ်ပါတယ်။

Show the commit details - Bug ဖြစ်စေတဲ့ Commit ရဲ့ အသေးစိတ်ကို ကြည့်ခြင်း

ရှာတွေ့ထားတဲ့ Commit ကို အသေးစိတ်ကြည့်ဖို့—

```
git show
```

ကို အသုံးပြုပါတယ်။

commit 31ed915660c42a00aa30b51520a16e3f48201299

Author: sandbox <sandbox@example.com>

main.sh

လို့ ပြပါတယ်။

Find the actual bug - တကယ့် Bug ကို ရှာဖွေခြင်း

git show နဲ့ Commit ရဲ့ Code ပြောင်းလဲမှုကို ကြည့်တဲ့အခါ—

```
-echo $(expr $1 + $2)
```

```
+echo $(expr $1 - $2)
```

ဖြစ်နေတာကို တွေ့ရပါတယ်။

မူလ Code က—

```
echo $(expr $1 + $2)
```

ဖြစ်ပြီး Bug ဖြစ်စေတဲ့ Commit မှာ—

```
echo $(expr $1 - $2)
```

လို့ ပေါင်းခြင်း (+) ကို နှုတ်ခြင်း (-) အဖြစ် ပြောင်းထားတာ ဖြစ်ပါတယ်။

အဓိက မှတ်ရန်

Commit History



git bisect start



Good Commit ————— Bad Commit



အလယ် Commit ကို Test



Good / Bad သတ်မှတ်



ထပ်မံ Test



Bug ဖြစ်စေတဲ့ Commit ရှာတွေ့

အဓိက Command တွေက—

git bisect start

→ Bisect စတင်ရန်

git bisect good <commit>

→ Bug မဖြစ်သေးတဲ့ Commit ကို သတ်မှတ်ရန်

git bisect bad <commit>

→ Bug ဖြစ်နေတဲ့ Commit ကို သတ်မှတ်ရန်

git show

→ ရှာတွေ့တဲ့ Commit ရဲ့ ပြောင်းလဲမှုကို စစ်ဆေးရန်

Bisect = Commit တစ်ခုချင်းစီကို အစကနေ လိုက်စစ်မယ့်အစား Good/Bad အပိုင်းတွေ ခွဲပြီး Bug ဖြစ်စေတဲ့ Commit ကို မြန်မြန်ရှာဖွေတဲ့ နည်းလမ်း ဖြစ်ပါတယ်။

5.4 Partial checkout — လိုအပ်သော အပိုင်းကိုသာ Checkout လုပ်ခြင်း

Selectively checkout only some directories -

(Repository ထဲက လိုအပ်တဲ့ Directory တွေကိုသာ Checkout လုပ်ခြင်း)

တစ်ခါတလေ Repository တစ်ခုလုံးကို Working Tree ထဲ Checkout လုပ်စရာမလိုဘဲ လိုအပ်တဲ့ Directory အချို့ကိုပဲ

Checkout လုပ်ချင် နိုင်ပါတယ်။

ဥပမာ Remote Repository ထဲမှာ—

```
.
├── go.mod
├── main.go
├── products
│   └── products.go
└── users
    └── users.go
```

ဆိုပြီး products နဲ့ users Directory နှစ်ခု ရှိပါတယ်။

ဒီထဲက root နဲ့ users Directory ကိုပဲ Checkout လုပ်ပြီး products ကို မ Checkout လုပ်ဘဲထားမယ် ဖြစ်ပါတယ်။

Clone without checking out the working tree -

(Working Tree ကို မ Checkout လုပ်ဘဲ Repository ကို Clone လုပ်ခြင်း)

အရင်ဆုံး Repository ကို Clone လုပ်ပေးမယ့် Working Tree ကို မ Checkout လုပ်ပါဘူး။

```
git clone --no-checkout /tmp/remote.git /tmp/repo
```

ပြီးရင် Repository Directory ထဲကို ဝင်ပါ။

```
cd /tmp/repo
```

Clone အောင်မြင်ရင်—

Cloning into '/tmp/repo'...

done.

လို့ ပြပါလိမ့်မယ်။

Enable sparse checkout -

(လိုအပ်တဲ့အပိုင်းကိုသာ Checkout လုပ်နိုင်အောင် Sparse Checkout ကို ဖွင့်ခြင်း)

```
git sparse-checkout init --cone
```

ကို အသုံးပြုပြီး Sparse Checkout ကို ဖွင့်ပါ။

Select the directory - Checkout လုပ်မယ့် Directory ကို သတ်မှတ်ခြင်း

ဒီဥပမာမှာ users Directory ကိုသာ ရွေးချယ်ပါတယ်။

```
git sparse-checkout set users
```

ဒီလိုလုပ်လိုက်တဲ့အခါ Git က users Directory ကိုသာ Working Tree ထဲ Checkout လုပ်ပေးမှာ ဖြစ်ပါတယ်။

Checkout the branch - Branch ကို Checkout လုပ်ခြင်း

ပြီးရင် main Branch ကို Checkout လုပ်ပါ။

```
git checkout main
```

အောင်မြင်ရင်—

Your branch is up to date with 'origin/main'.

လို့ ပြပါလိမ့်မယ်။

Check the working tree - Working Tree ထဲမှာ ဘာတွေရှိလဲ စစ်ဆေးခြင်း

tree

ရလဒ်က—

```
.
├── go.mod
├── main.go
└── users
    └── users.go
```

ဖြစ်ပါတယ်။

ဒီမှာ—

- go.mod → Checkout လုပ်ထားပါတယ်
- main.go → Checkout လုပ်ထားပါတယ်
- users/ → Checkout လုပ်ထားပါတယ်
- products/ → **Checkout မလုပ်ထားပါ**

ဒါကြောင့် Working Tree ထဲမှာ **Root** နဲ့ **users Directory** ပဲ ရှိနေပါတယ်။

အဓိက မှတ်ရန်

Remote Repository

```
├── products/
│   └── products.go
└── users/
    └── users.go
```

↓

Sparse Checkout

↓

Local Working Tree

```
├── go.mod
├── main.go
└── users/
    └── users.go
```

အဓိက Command တွေက—

`git clone --no-checkout /tmp/remote.git /tmp/repo`

→ Working Tree ကို မ Checkout လုပ်ဘဲ Repository ကို Clone လုပ်ရန်။

`git sparse-checkout init --clone`

→ Sparse Checkout ကို စတင်အသုံးပြုရန်။

`git sparse-checkout set users`

→ users Directory ကိုသာ Checkout လုပ်ရန်။

Partial checkout = Repository တစ်ခုလုံးကို Working Tree ထဲ ထည့်မယ့်အစား လိုအပ်တဲ့ Directory / အပိုင်းကိုသာ

ရွေးပြီး Checkout လုပ်ခြင်း ဖြစ်ပါတယ်။

5.5 Partial clone — Repository ၏ လိုအပ်သော Data ကိုသာ Clone လုပ်ခြင်း

Why use partial clone? - Repository တစ်ခုလုံးကို Download မလုပ်ဘဲ လိုအပ်တဲ့ Data ကိုသာ ရယူခြင်း

အရင် **Partial checkout** မှာ Repository ထဲက လိုအပ်တဲ့ Directory တွေကိုပဲ Working Tree ထဲ Checkout လုပ်ခဲ့ပါတယ်။

ဒါပေမယ့် အဲဒီနည်းလမ်းမှာ **Repository တစ်ခုလုံးကို Clone လုပ်ထားဆဲ** ဖြစ်ပါတယ်။

Repository ကိုယ်တိုင်က အလွန်ကြီးမားနေမယ်ဆိုရင်—ဥပမာ **History ရှည်လျားတာ** ဒါမှမဟုတ် **Binary File အကြီးတွေ ပါဝင်တာ**—Clone လုပ်တဲ့အချိန်မှာ Download လုပ်ရမယ့် Data များပြီး အချိန်ကြာနိုင်သလို Network Traffic လည်း များနိုင်ပါတယ်။

Clone without downloading all blobs - File Contents (Blobs) အားလုံးကို မ Download လုပ်ဘဲ Clone လုပ်ခြင်း

Data Download ပမာဏကို လျှော့ချဖို့—

```
git clone --filter=blob:none file:///tmp/remote.git
```

ကို အသုံးပြုနိုင်ပါတယ်။

ဒီနည်းလမ်းမှာ—

- **Commits** → Download လုပ်ပါတယ်
- **Trees (Directories)** → Download လုပ်ပါတယ်
- **Blobs (File Contents)** → အစပိုင်းမှာ မ Download လုပ်ပါ

ဖြစ်ပါတယ်။

လိုအပ်တဲ့ File Content ကို နောက်ပိုင်းမှာ အသုံးပြုတဲ့အခါ Git က **လိုအပ်သလို Lazy Fetch** လုပ်ပေးနိုင်ပါတယ်။

Clone commits only - Commit တွေကိုသာ Download လုပ်ခြင်း

ပိုပြီး Data လျှော့ချချင်ရင်—

```
git clone --filter=tree:0 file:///tmp/remote.git
```

ကို အသုံးပြုနိုင်ပါတယ်။

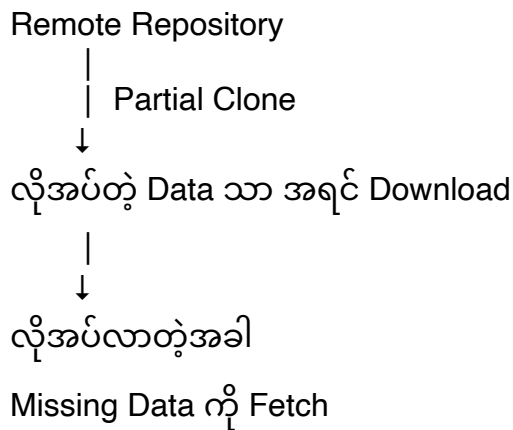
ဒီနည်းလမ်းမှာ—

- **Commits** → Download လုပ်ပါတယ်
- **Trees (Directories)** → အစပိုင်းမှာ မ Download လုပ်ပါ
- **Blobs (File Contents)** → အစပိုင်းမှာ မ Download လုပ်ပါ

ဖြစ်ပါတယ်။

Lazy fetch - လိုအပ်လာတဲ့အချိန်မှ Data ကို ရယူခြင်း

Partial clone ရဲ့ အဓိကအချက်က Data အားလုံးကို အစကတည်းက Download လုပ်ထားစရာမလိုဘဲ **လိုအပ်လာတဲ့အချိန်မှာ Git က Missing Data ကို နောက်မှ ရယူပေးနိုင်ခြင်း** ဖြစ်ပါတယ်။



Remote Server Support လိုအပ်ခြင်း

Remote Server က Partial Clone ကို Support လုပ်ရမယ်။

ဥပမာ GitHub က Partial Clone ကို Support လုပ်ပါတယ်။

အဓိက မှတ်ရန်

`git clone --filter=blob:none file:///tmp/remote.git`

→ **Commits + Trees** ကို ရယူပြီး **Blobs (File Contents)** ကို လိုအပ်တဲ့အချိန်မှ ရယူရန်။

`git clone --filter=tree:0 file:///tmp/remote.git`

→ **Commits** ကိုသာ အရင်ရယူပြီး **Trees + Blobs** ကို လိုအပ်တဲ့အချိန်မှ ရယူရန်။

Partial checkout

→ Repository တစ်ခုလုံး Clone လုပ်ထားပြီး

လိုအပ်တဲ့ Directory ကိုသာ Checkout

Partial clone

→ Clone လုပ်တဲ့အချိန်ကတည်းက

Download လုပ်မယ့် Data ပမာဏကို လျှော့ချ

Partial clone = Repository ကြီးမားတဲ့အခါ Clone လုပ်ချိန်မှာ Download လုပ်ရမယ့် Data ပမာဏကို လျှော့ချပြီး

လိုအပ်တဲ့ Data ကို နောက်မှ ရယူတဲ့ နည်းလမ်း ဖြစ်ပါတယ်။